Assignment 05 AOA

Camilo Vasquez

September 21, 2026

GitHub Repository Link:
`https://github.com/Camimilo-11/Analysis-of-algorithms/blob/main/Assignments/Assignment05/lab05_solutions.cpp`

Problem 1: Insertion Sort for Alphabetic Characters

Analysis & Adjustments:

- **Algorithm Logic:** No structural changes are required when switching from integers to characters. In C++, `char` variables are stored as ASCII values ($'A' = 65, 'B' = 66, 'a' = 97$, etc.). Standard relational operators ($>, <$) compare these ASCII integers directly.
- **Case Sensitivity:** Standard ASCII orders all uppercase letters ($'A'$–$'Z'$) before lowercase letters ($'a'$–$'z'$). For case-insensitive sorting, apply `tolower()` during comparisons.

Problem 2: Binary Search vs. Interpolation Search ($10,000,000$ Elements)

Comparison & Preconditions:

- **Precondition:** Both algorithms strictly require the input array to be pre-sorted.
- **Binary Search:** Uses a *Decrease-by-a-Constant-Factor* strategy ($m = \lfloor (l + r)/2 \rfloor$). Time complexity is $\Theta(\log_2 n)$ in all cases. For $N = 10^7$, it takes at most $\lceil \log_2(10^7) \rceil + 1 \approx 24$ comparisons.
- **Interpolation Search:** Uses a *Variable-Size-Decrease* strategy, estimating target index as $pos = l + \left\lfloor \frac{(key - A[l])(r-l)}{A[r]-A[l]} \right\rfloor$. Time complexity is $O(\log_2 \log_2 n)$ on uniformly distributed data (≈ 4.5 iterations for 10^7), but degrades to $O(n)$ in the worst case (non-uniform distributions).

Problem 3: Proof: Topological Ordering Exists $\iff$ Graph is a DAG

Proof ($\Rightarrow$): If G has a topological sort, then G is a DAG.

Proof. Suppose G has a topological ordering represented by a position function $pos(v)$ such that $(u, v) \in E \implies pos(u) < pos(v)$. Assume by contradiction that G contains a directed cycle $C = (v_1, v_2, \ldots, v_k, v_1)$. Applying the topological property around the cycle yields:

$$pos(v_1) < pos(v_2) < \cdots < pos(v_k) < pos(v_1) \implies pos(v_1) < pos(v_1),$$

which is a contradiction. Thus, G cannot contain directed cycles and must be a DAG. □

Proof ($\Leftarrow$): If G is a DAG, then G has at least one topological sort.

Proof. Lemma: Every finite DAG has at least one source (in-degree 0).
Induction on $n = |V|$:

- **Base Case ($n = 1$):** Trivial.
- **Inductive Step:** Assume true for DAGs with $k < n$ vertices. Let G be a DAG with n vertices. By the Lemma, G has a source v_1. Place v_1 first in the ordering and remove it along with its outgoing edges. The remaining subgraph $G' = G \setminus \{v_1\}$ is still a DAG with $n - 1$ vertices. By the inductive hypothesis, G' has a valid topological ordering $(v_2, \ldots, v_n)$. Concatenating yields $(v_1, v_2, \ldots, v_n)$, which is valid since v_1 had no incoming edges.

□

Problem 4: Maximum Distinct Topological Sorts in a DAG with n Vertices

A topological sort is a vertex permutation satisfying all edge precedence constraints. To **maximize** valid permutations, the edge set E must impose the minimum possible constraints.

This occurs when $E = \emptyset$ (a graph with n isolated vertices), which is trivially a DAG. With no edges, **every** permutation of the n vertices is a valid topological ordering.

$$\text{Maximum distinct solutions} = n!$$